

Procédure de test — Chaîne Catalogue -> UO (projet Train System)

Objet : valider la chaîne complète d'instanciation et de synchronisation des UO : génération du catalogue depuis le Word, création d'une UO en une commande, saisie ingénieur, synchronisation, KPI. **Durée estimée** : 45-60 minutes. **Tous les résultats attendus de cette procédure ont été vérifiés le 2026-06-12.**

Note ce que tu obtiens à chaque étape dans le classeur de suivi Validation-UO-TrainSystem.xlsx (statut OK/KO + remarques). En cas de KO, continue si possible — le bilan global compte plus qu'un blocage isolé.

Prérequis

1. Dépôt à jour : `git pull` dans le dossier SysEng.
2. Dépendances : `pip install -r requirements.txt` (une fois).
3. **Le fichier Word UO TS.docx** — il est **confidentiel** et n'est PAS sur GitHub : il doit t'être transmis par canal interne (mail/Teams). Note le chemin où tu l'enregistres.
4. Toutes les commandes se lancent **depuis la racine du dossier SysEng**.
5. Règle d'or : **toujours fermer le fichier Excel avant de lancer une commande**, et le rouvrir après pour constater le résultat.

Étape 1 — Générer le catalogue depuis le Word

```
python projet_TrainSystem/parse_catalogue.py "CHEMIN\VERS\UO TS.docx"
python projet_TrainSystem/build_catalogue.py
```

Attendu : - La 1re commande affiche un tableau de **10 lignes** (L09U1 -> L13U2) avec les comptes d'activités par UO. - La 2e affiche : [OK] Catalogue_UO_TrainSystem.xlsx – 10 UO types, 54 activités, 97 livrables, 88 données d'entrée. - Ouvre projet_TrainSystem\Catalogue_UO_TrainSystem.xlsx : 4 onglets (Index, Catalogue_Activites, Catalogue_Livrables, Catalogue_DonneesEntree). Dans Catalogue_Activites, filtre la colonne `uo_type` sur L09U1 : **13 lignes**.

Étape 2 — Tests négatifs (codes invalides)

```
python projet_TrainSystem/creer_uo.py CODE-INVALIDE
python projet_TrainSystem/creer_uo.py L99U9-TEST
```

Attendu : deux messages d'erreur clairs (pas de plantage Python) : - [ERR] Code invalide 'CODE-INVALIDE' – attendu : L09U1-PROJET-SYSTEME - [ERR] UO type 'L99U9' absent du catalogue. Disponibles : L09U1, L09U2, ...

Étape 3 — Créer l'UO de test

```
python projet_TrainSystem/creer_uo.py L09U1-TEST01-CLIM --se "Ton Nom" --heures 240 --projet "Projet Test"
```

Attendu : [OK] ...L09U1-TEST01-CLIM.xlsx créé. Ouvre-le et vérifie : - **11 feuilles** : General, Description_Besoin, Donnees_Entree, Activites, Livrables, OIL, KPI, Dashboard, Planning, Orga, _Manifeste ; - **General** : projet, système, ton nom, **240** en Heures vendues ; - **Activites** : **13 lignes** pré-remplies (applicable=OUI, avancement=0, heures_allouees calculées ≈ 18,46 h chacune) ; - **Description_Besoin** : le cahier des charges complet du type L09U1 ; - **Livrables** : 21 lignes - **Donnees_Entree** : 21 lignes ; - **OIL** : 1 ligne d'exemple **PO-000** ; - Les listes déroulantes fonctionnent (colonne statut d'Activites, etc.).

Ferme le fichier.

Étape 4 — Synchronisation à blanc (UO vierge)

```
python -m src store clear
python scripts/valider_un.py projet_TrainSystem/L09U1-TEST01-CLIM.xlsx
```

Attendu (console) : PUSH = 8, BIND = 7, [OK] Execution terminee sans erreur. — et 8 clés uo.L09U1-TEST01-CLIM. * listées dans le store. Rouvre le fichier : onglet **KPI** -> Avancement **0** ; Points ouverts **0** ; Points fermés **1** (la ligne d'exemple) ; Santé **VERT**. Referme.

Étape 5 — Saisie ingénieur (scénario imposé)

Ouvre le fichier et saisis **exactement** ceci :

Feuille Activites (les lignes du tableau, ligne 2 = première activité) : | Ligne | applicable | statut | avancement | heures_consommees | |---|---|---|---| | 2 | OUI | EN_COURS | **50** | **10** | | 3 | OUI | TERMINEE | **100** | **20** | | 5 | **NON** | — | — | — | | toutes les autres | OUI (inchangé) | inchangé | 0 | 0 |

Feuille OIL — remplace la ligne d'exemple et ajoute 2 lignes (étire le tableau si besoin en tirant la poignée en bas à droite) : | id | titre | en_action | criticite | statut | |---|---|---|---| | PO-001 | Question expert normes | EXPERT | HAUTE | OUVERT | | PO-002 | Attente retour fournisseur | FOURNISSEUR | MOYENNE | OUVERT | | PO-003 | Accès outil obtenu | SE | BASSE | CLOS |

Renseigne aussi une date d'ouverture et une ligne de **journal** sur chaque point (texte libre). **Enregistre et ferme.**

Étape 6 — Synchronisation et vérification des KPI

```
python scripts/valider_un.py projet_TrainSystem/L09U1-TEST01-CLIM.xlsx
```

Attendu (console) : zéro erreur. Rouvre le fichier, onglet **KPI** :

KPI	Valeur attendue	Pourquoi
Avancement UO (%)	12,5	(50+100+0×10) ÷ 12 activités applicables
Heures consommées	30	10 + 20
Points ouverts	2	PO-001, PO-002
Points fermés	1	PO-003
Points critiques ouverts	1	PO-001 (HAUTE)

Dont balle chez fournisseur	1	PO-002
Dont balle chez expert	1	PO-001
Reste à faire total (h)	210	formule Excel (recalculée à l'ouverture)
EAC	240	30 consommées + 210 RAF
Dérive à terminaison	0	EAC = heures vendues
Santé	ROUGE	un point critique est ouvert

L'onglet `Dashboard` doit refléter les mêmes valeurs (cartes).

[i] Les lignes « Reste à faire », « EAC », « Dérive », « Santé » sont des formules Excel : elles s'actualisent à l'ouverture du fichier, pas à la sync.

Étape 7 — Règles de qualité (VALIDATE)

- Ouvre le fichier, mets l'avancement de la **ligne 2 à 150**. Enregistre, ferme.
- Relance la sync.

Attendu : la sync se termine **en erreur**, avec un message explicite : `[ERR] VALIDATE $actifs.avancement : RANGE(0.0,100.0) : 1 valeur(s) hors plage ([150])`

- Remets **50**, enregistre, ferme, relance : **zéro erreur**, KPI inchangés (mêmes valeurs qu'à l'étape 6 — la sync est répétable).

Étape 8 — Le store (ce que verront les cockpits)

```
python -m src status --prefix uo.
```

Attendu : les 8 clés de l'UO avec leurs valeurs (`avancement = 12.5`, `activites = [table: 12 lignes]...`). C'est ce qui alimentera le cockpit ingénieur et le dashboard métier.

Étape 9 — Bilan libre (le plus important)

Mets-toi 10 minutes dans la peau d'un ingénieur qui reçoit ce fichier : crée une 2e UO d'un autre type (ex. `L11U1-TEST02-FREIN`), remplis-la comme tu le ferais en vrai, et note **tout ce qui te gêne** : colonnes manquantes ou inutiles, libellés pas clairs, saisies pénibles, KPI faux ou absents, idées. Ces remarques valent plus que les OK/KO — elles feront la v2.

Récapitulatif des commandes

```
git pull
pip install -r requirements.txt
python projet_TrainSystem/parse_catalogue.py "CHEMIN\UO TS.docx" # 1. catalogue (json)
python projet_TrainSystem/build_catalogue.py # 1. catalogue (xlsx)
python projet_TrainSystem/creer_uo.py CODE --se "Nom" --heures N # 3. créer une UO
python -m src store clear # remise à zéro
```

```
python scripts/valider_un.py projet_TrainSystem/CODE.xlsx      # synchroniser
python -m src status --prefix uo.                             # inspecter le store
```